SIMATS ENGINEERING

Assessment Tool 3: Reverse Engineering Task — Answer Key

Course Name	Cloud Computing and Big Data Analytics
Course Code	CSA15
Course Outcome	CO4 — Analyze big data characteristics and challenges across domains including security and application aspects (BL4)
Weightage	10%
Bloom's Level	Articulation (Level 4)
Question Count	5
Total Marks	30

Q1. A recommendation engine processes user clicks, search queries, and purchases from multiple devices. Reverse engineer the probable data types, ingestion flow, and analytics stages.

Answer

1. Probable Data Types and Sources

A recommendation engine of this kind almost certainly ingests three broad categories of data. Clickstream events (page views, scroll depth, hover time, add-to-cart actions) are semi-structured, arriving as JSON events with a user/session ID, timestamp, device ID, and event metadata. Search queries are short text strings accompanied by contextual metadata (filters applied, result-click position), making them semi-structured/text data that typically needs tokenization before analysis. Purchase data is structured, coming from an order-management or ERP system with fixed fields such as order ID, SKU, price, and quantity. Because the system explicitly spans multiple devices, there is also almost certainly a device/session-linking layer — a probabilistic or deterministic identity-resolution component — that stitches a single user's behavior together across mobile app, desktop web, and possibly in-store point-of-sale systems.

2. Probable Ingestion Flow

- Event collection SDKs embedded in the web/app front ends emit click and search events to a lightweight collector endpoint, most likely fronted by a message broker such as Apache Kafka or a managed equivalent (Amazon Kinesis), chosen because recommendation freshness depends on low-latency ingestion of user behavior.
- Purchase/order data, being transactional and already structured, is more likely captured via change-data-capture (CDC) from the order database (e.g., Debezium reading MySQL/PostgreSQL binlogs) rather than a custom event SDK, since it must remain perfectly consistent with the financial system of record.
- A stream-processing layer (Kafka Streams, Flink, or Spark Structured Streaming) sits immediately downstream of the broker to perform light transformation — deduplication of repeated click events, session windowing, and identity stitching across devices — before the data lands in storage.
- Raw and lightly processed events are persisted into a distributed data lake (HDFS or S3) in a partitioned, columnar format (Parquet/ORC), which is the standard pattern for systems that must support both fast streaming reads and later large-scale batch model training.

3. Probable Analytics / Model Stages

- Feature engineering: batch Spark jobs periodically aggregate raw events into user-level and item-level features — recency/frequency of clicks, category affinity scores, co-purchase patterns — stored in a feature store for reuse across models.
- Candidate generation: a collaborative-filtering or embedding-based model (e.g., matrix factorization, two-tower neural network) narrows millions of catalog items down to a few hundred plausible candidates per user, since scoring the entire catalog in real time would be computationally infeasible.
- Ranking: a supervised ranking model (gradient-boosted trees or a deep ranking network) reorders the candidate set using richer features — including the very recent clickstream signals — to produce the final top-N recommendations.
- Online serving layer: a low-latency key-value store (Redis, DynamoDB, or Cassandra) caches precomputed recommendations and feature vectors so the website can render suggestions within milliseconds of a page request.
- Feedback loop: new clicks/purchases resulting from shown recommendations are fed back into the pipeline, closing the loop for continuous (often daily) model retraining.

4. Likely Design Decisions and Rationale

The separation of a fast streaming path (for near-real-time personalization signals) from a slower batch path (for deep feature engineering and model retraining) is a classic Lambda/Kappa-style architecture decision. It is the most defensible design because clickstream volume is far too high for synchronous processing on every request, yet recommendation quality benefits from very recent behavioral signals, so a hybrid path balances latency against modeling depth. Using a data lake with columnar storage instead of a single relational warehouse is a strong inference

because model training over months of interaction history at this scale would overwhelm a traditional RDBMS both in storage cost and scan performance.

5. Technical Evidence Supporting This Inference

The requirement to unify clicks, queries, and purchases across multiple devices is itself evidence of an identity-resolution and event-streaming layer, since these signals originate from structurally different systems (web analytics vs. transactional databases) that would not naturally share a common key without deliberate engineering. The need for the engine to reflect very recent behavior (a user searching moments ago) is evidence of a streaming ingestion and low-latency serving layer rather than a purely batch pipeline, because batch-only systems typically refresh on the order of hours, which would be perceptibly stale for click-driven personalization.

6. Challenges This Design Most Likely Addresses

- Cross-device identity fragmentation: without deliberate stitching, the same shopper on mobile and desktop would appear as two unrelated users, silently halving the effective training signal available to the model — this is almost certainly why an identity-resolution step is placed early in the pipeline rather than left to the modeling stage.
- Cold-start items and users: a pure collaborative-filtering candidate generator struggles for brand-new users or products with no interaction history, so the design most likely supplements it with content-based features (category, price band, text similarity of search queries) to produce reasonable candidates even before behavioral data accumulates.
- Catalog-scale scoring latency: ranking every catalog item for every request in real time would be computationally infeasible at e-commerce scale, which is precisely why a two-stage candidate-generation-then-ranking funnel is the standard and most defensible pattern rather than a single end-to-end scoring pass.
- Staleness versus load trade-off: refreshing the entire feature store on every click would overwhelm the batch layer, so periodic (e.g., hourly/daily) batch recomputation combined with lightweight real-time feature overlays is the most plausible compromise.

7. Trade-offs and Alternative Considerations

An alternative, simpler design would run everything as a single nightly batch job that recomputes recommendations once per day. This would be far cheaper to build and operate, but it would fail the implied requirement that the engine reacts to in-session behavior (e.g., a user who just searched for 'running shoes' seeing relevant items later in the same session), so it is a less likely fit for the described system. Similarly, an architecture that scores the full catalog per request using only online computation would guarantee freshness but would not scale to millions of concurrent users without prohibitive infrastructure cost — which is why the candidate-generation-plus-ranking funnel, backed by a precomputed cache, remains the more defensible inference despite being more complex to engineer.

Q2. A ride-sharing company displays live traffic heatmaps and demand forecasts. Reverse engineer the likely distributed storage and processing backbone behind the system.

Answer

1. Probable Data Sources

The system almost certainly consumes continuous GPS pings from millions of driver and rider mobile apps (latitude, longitude, timestamp, speed), ride-request events (pickup/drop-off location, timestamp, fare estimate), and possibly third-party traffic/road-condition feeds. GPS pings are high-frequency, small, semi-structured events, while ride requests are structured transactional records. Given the scale implied — a live, city-wide heatmap — the underlying volume is very high velocity and geographically distributed, which strongly shapes the architecture.

2. Probable Distributed Storage Backbone

- A distributed message broker (Kafka) is almost certainly the entry point, partitioned by geographic region or city ID, so that location-based consumers can subscribe only to relevant partitions instead of the entire global stream.
- Hot, recent location data (needed for the live heatmap) is likely held in an in-memory or low-latency store — Redis with geospatial indexing (GEOADD/GEORADIUS) or Apache Cassandra with geohash-based partition keys — because sub-second read latency is required to render a moving heatmap.
- Cold, historical trip and location data used for demand forecasting is offloaded to a distributed file system (HDFS or S3) in partitioned Parquet files, partitioned by date and region, since forecasting models need months of historical patterns rather than the current second's data.
- A geospatial indexing scheme (geohashing or an H3 hexagonal grid, as used by Uber's own published architecture) is a highly probable design choice, since it allows the system to bucket GPS points into discrete cells that can be aggregated and rendered as a heatmap far more efficiently than raw lat/long comparisons.

3. Probable Processing Backbone

- Stream processing (Spark Structured Streaming or Flink) continuously aggregates raw GPS pings into per-cell, per-time-window density counts (e.g., every 30 seconds) that directly feed the live heatmap — this windowed aggregation pattern is a standard response to high-velocity geospatial streams.
- A separate batch pipeline (Spark on the historical HDFS/S3 store) trains time-series or ML-based demand-forecasting models (e.g., gradient boosting or LSTM-based models) using historical ride requests, weather, and calendar/event features, retrained on a daily or hourly schedule.
- A serving layer exposes both the live aggregated heatmap cells and the forecast outputs through a low-latency API, likely backed by the same in-memory store used for hot data, so the mobile/web front end can poll or subscribe to updates smoothly.

4. Likely Design Decisions and Rationale

Splitting storage into a 'hot path' (in-memory/geo-indexed store for the live view) and a 'cold path' (distributed file system for historical training data) is the most defensible inference because the two use cases have opposing requirements: the heatmap needs millisecond freshness on a small recent window, while forecasting needs deep historical depth but can tolerate periodic (not sub-second) refresh. Geohash/H3-style spatial partitioning is a reasonable inference because naively storing raw coordinates would make range queries over a map viewport computationally expensive at ride-sharing scale.

5. Technical Evidence Supporting This Inference

The word 'live' in the heatmap requirement is direct evidence of a streaming, low-latency architecture rather than a batch-only one, since a batch job refreshed hourly could not be described as live. The requirement for 'demand forecasts' as a distinct feature from the heatmap is evidence of a separate, model-driven batch pipeline, because forecasting inherently requires trend analysis over historical data that a purely streaming system does not retain in the same low-latency store.

6. Challenges This Design Most Likely Addresses

- GPS noise and sparsity: raw location pings are noisy (signal drift, tunnels, urban canyons) and arrive at uneven intervals per device, so the streaming aggregation layer most likely applies smoothing/snapping (e.g., map-matching to road segments) before the data is usable for a clean heatmap.

- Uneven geographic load: city centers generate vastly more pings than suburban areas, so partitioning by geohash/H3 cell rather than by raw device ID is almost certainly necessary to avoid a small number of 'hot' partitions overwhelming individual stream-processing nodes.
- Forecast accuracy during anomalies: demand during unusual events (concerts, storms) deviates sharply from historical patterns, so the forecasting model most likely incorporates external event/weather features rather than relying purely on historical ride counts, which is a common augmentation in production demand-forecasting systems.
- Balancing freshness with system load: recomputing the full heatmap on every single incoming ping would be wasteful, so windowed micro-batch aggregation (e.g., every 15–30 seconds) is the most defensible middle ground between true per-event updates and stale periodic refreshes.

7. Trade-offs and Alternative Considerations

A simpler alternative would store all GPS pings directly in a general-purpose relational database and compute the heatmap with on-demand SQL aggregation. This would be easier to build initially, but at ride-sharing scale (millions of pings per minute across a city) it would not sustain the read/write throughput or the sub-second query latency the live heatmap requires, making a geo-indexed, streaming-first design the far more defensible inference. Likewise, forecasting could theoretically be done with simple historical averages instead of a trained ML model, but this would perform poorly around anomalies and special events, so a model-based approach incorporating external signals is the more plausible and higher-quality design choice actually used in production systems of this kind.

Q3. A fraud detection dashboard flags suspicious card transactions in near real time. Reverse engineer the probable big data security, streaming, and alerting mechanisms used.

Answer

1. Probable Data Flow and Streaming Mechanism

Card transaction events (cardholder ID, merchant, amount, location, timestamp) are generated at the point of sale or payment gateway and must reach the fraud engine within milliseconds to seconds, since 'near real time' flagging implies the system can intervene before or immediately after a transaction clears. This strongly implies a streaming ingestion layer — most likely Apache Kafka — acting as a durable, ordered buffer between the payment gateway and the fraud-scoring engine, with a stream processor (Flink or Spark Structured Streaming) consuming from it continuously rather than on a batch schedule.

2. Probable Processing and Scoring Pipeline

- Stateful stream processing maintains rolling aggregates per cardholder (transaction count in the last 5/30/60 minutes, average spend, distance between consecutive transaction locations), because fraud signals like 'two transactions in different cities within minutes' require state that persists across events rather than evaluating each transaction in isolation.
- A trained ML model (likely gradient boosting, isolation forest, or a neural network) scores each transaction against these rolling features in-stream, producing a fraud-probability score with a latency budget in the tens to hundreds of milliseconds.
- A rules engine likely runs alongside the ML model as a complementary layer, encoding hard business rules (e.g., transaction amount above a threshold in a high-risk country) that must trigger instantly regardless of model confidence — a common hybrid design in production fraud systems.
- Flagged transactions are pushed to an alerting topic/queue, which feeds both the analyst-facing dashboard (via a real-time push mechanism such as WebSockets or server-sent events) and an automated action layer that may hold or decline the transaction.

3. Probable Big Data Security Mechanisms

- Encryption in transit (TLS) between the payment gateway, Kafka, and the scoring service, since raw card data traversing a distributed pipeline is a prime target for interception.
- Tokenization of the primary account number (PAN) at the earliest possible point, so that downstream streaming and storage components never handle the raw card number — this is standard PCI-DSS-driven design and is a very strong inference given the domain.
- Encryption at rest (AES-256) for any persisted transaction history used for feature aggregation or model retraining, with keys managed by a dedicated KMS rather than embedded in application code.
- Strict role-based access control on the analyst dashboard, since fraud analysts need to see flagged-transaction context but should not have broad access to full cardholder PAN or unrelated customer data.
- Immutable audit logging of every flag, override, and analyst action, both for regulatory compliance (PCI-DSS) and for feeding back into future model evaluation.

4. Likely Design Decisions and Rationale

The combination of a rules engine and an ML model is a defensible inference because pure ML systems can be slow to react to brand-new fraud patterns (concept drift), while pure rules systems miss subtle statistical anomalies — production fraud systems in the industry typically hybridize both for exactly this reason. Tokenizing PAN data at ingestion is the most defensible security decision because it minimizes the scope of PCI-DSS compliance across the rest of the distributed pipeline, reducing risk if any downstream component is compromised.

5. Technical Evidence Supporting This Inference

The phrase 'near real time' is direct evidence against a batch architecture and for a streaming, stateful-processing design, since a batch job evaluated hourly could not credibly flag fraud before financial damage occurs. The fact that the system flags patterns (implicitly, unusual behavior relative to a baseline) rather than single-field rule violations is evidence of stateful, per-entity aggregation rather than stateless, per-record filtering.

6. Challenges This Design Most Likely Addresses

- Class imbalance: genuine fraud is a tiny fraction of all transactions, so the ML scoring model most likely uses techniques such as resampling, anomaly-detection framing, or cost-sensitive learning rather than plain classification, since a naively trained model would otherwise default to predicting 'not fraud' almost always.
- Latency versus accuracy trade-off: a deeper, more accurate model may take longer to score, so the design most plausibly tiers its checks — instant rule-based blocking for extreme cases, followed by an ML score for borderline transactions — balancing speed against detection quality.
- Concept drift: fraud tactics evolve quickly, so the pipeline most likely includes a feedback loop where confirmed fraud/non-fraud outcomes (from analyst review or chargebacks) are used to periodically retrain the model, since a static model would degrade in accuracy over time.
- False-positive cost: blocking a legitimate transaction damages customer trust, so the dashboard and alerting layer most likely present a confidence score and supporting evidence to human analysts rather than fully automating every decision, especially for medium-risk scores.

7. Trade-offs and Alternative Considerations

An alternative design relying solely on static, hand-written rules (e.g., 'block if amount > $5000') would be simpler to build and fully explainable, but it would be easily circumvented by fraudsters staying just under thresholds and would miss subtler statistical anomalies — this is why a hybrid rules-plus-ML approach is the more defensible inference. A purely batch fraud-detection job run every few hours would be considerably cheaper to operate than a stateful streaming pipeline, but it directly conflicts with the 'near real time' requirement in the prompt, since by the time a batch job flags a fraudulent charge the funds may already have settled — making the streaming, stateful architecture the far more plausible design given the stated constraint.

Q4. A social media platform stores videos, text comments, hashtags, and user engagement metrics. Reverse engineer how structured and unstructured data are probably managed together.

Answer

1. Probable Data Type Breakdown

This platform spans the full spectrum of the data-variety dimension. Videos are large binary, unstructured objects. Text comments and hashtags are semi-structured/text data with associated metadata (author, timestamp, parent post). Engagement metrics (likes, shares, view counts, watch-time) are highly structured, numeric, and update at extremely high frequency. Managing all three together under one platform almost certainly requires a polyglot storage strategy rather than a single database technology, since no single system is simultaneously optimal for large binary blobs, searchable text, and high-write-throughput counters.

2. Probable Storage Architecture

- Video files are almost certainly stored in an object storage system (Amazon S3, or an HDFS-backed equivalent) rather than a database, with only a lightweight pointer/URL and metadata (duration, resolution, upload time) kept in a structured store — storing large binaries directly inside a database would be highly inefficient at this scale.
- A Content Delivery Network (CDN) almost certainly sits in front of the object store to serve videos with low latency globally, since a social platform's traffic pattern (many reads of popular videos) is a textbook CDN caching use case.
- Text comments and hashtags are most likely stored in a NoSQL document or wide-column store (e.g., Cassandra or MongoDB) that can handle a very high write rate of small records and flexible schema evolution (new comment features, reactions, etc.) without costly schema migrations.
- A dedicated search index (Elasticsearch or Solr) is a strong inference for hashtag and comment discoverability, since full-text and hashtag search over billions of comments cannot be efficiently served directly from a general-purpose NoSQL store.
- Engagement metrics (like/view counters) are most plausibly maintained in a fast, in-memory or counter-optimized store (Redis counters, or a wide-column store with atomic increment support) because these values are updated at extremely high frequency and must be read back with low latency on every page view.

3. Probable Integration / Unification Layer

- A metadata catalog or graph layer likely links a single 'post' entity to its video object reference, its comment thread ID, its hashtag tags, and its engagement counters, allowing the application layer to assemble a complete post view from multiple underlying stores in one composite query.
- Asynchronous, event-driven updates (via Kafka) most likely propagate engagement events (a new like) to the counters, to the search index (for trending hashtag ranking), and to any real-time notification service, rather than each subsystem being updated synchronously within a single transaction — full ACID transactions across all these stores would be impractical at this scale.
- Periodic batch jobs (Spark) likely aggregate raw engagement events into daily/weekly analytics tables (used for trending content, creator dashboards) stored in a data warehouse, separate from the real-time counters used for live display.

4. Likely Design Decisions and Rationale

The decision to keep video binaries out of the primary database and instead store only references is a defensible, near-universal design choice at this scale, since it lets the object store and CDN independently scale for bandwidth-heavy reads without burdening the metadata database. Using eventual consistency (via asynchronous event propagation) between the like-counter, the search index, and the analytics tables is a reasonable inference because strict real-time consistency across four separate systems would introduce unacceptable write latency for a feature as high-frequency as 'likes'.

5. Technical Evidence Supporting This Inference

The explicit mention of four structurally different data types (video, text, hashtags, metrics) in a single platform is itself strong evidence of a polyglot-persistence design, since a single database technology optimized for one of these

would perform poorly for the others. The fact that hashtags must be discoverable (implied by their purpose) is evidence of a dedicated search/indexing layer rather than reliance on the primary transactional store for text search.

6. Challenges This Design Most Likely Addresses

- Write amplification from viral content: a single popular video or trending hashtag can generate an extreme, sudden spike in engagement writes, so the counter layer most likely uses a distributed, shardable increment mechanism rather than a single row lock in a traditional database, which would become a severe bottleneck under a viral spike.
- Read-heavy video delivery at global scale: since videos are watched far more often than uploaded, the CDN-fronted object storage design most likely exists specifically to offload this heavily skewed read traffic away from origin servers and databases entirely.
- Consistency versus availability trade-off: strict, immediate consistency of a like-count across all replicas is less important than the platform staying available and responsive, so eventual consistency (a brief delay before a like count fully propagates) is a defensible and commonly accepted trade-off in this domain.
- Schema evolution: social platforms frequently add new interaction types (reactions, shares, saves), so a flexible NoSQL schema for comments/metadata is a more plausible choice than a rigid relational schema that would require frequent, disruptive migrations.

7. Trade-offs and Alternative Considerations

A simpler alternative would store everything — video references, comments, and counters — in a single relational database. This would simplify consistency and querying, but it would not scale to the write throughput of billions of daily likes/comments or the storage economics of video, making the polyglot approach the more defensible inference despite its added integration complexity. Similarly, updating the search index synchronously with every new comment would guarantee instantly up-to-date search results, but would add latency to every comment submission; asynchronous, event-driven indexing is the more plausible choice because it favors a fast user-facing write path over immediate search consistency, an acceptable trade-off for this kind of platform.

Q5. A manufacturing IoT platform collects machine logs and maintenance events from many plants. Reverse engineer the likely integration tools, storage strategy, and analytics path.

Answer

1. Probable Data Sources and Types

Machine logs from industrial equipment are typically high-frequency time-series telemetry (temperature, vibration, RPM, pressure) emitted by PLCs/sensors, while maintenance events (work orders, part replacements, technician notes) are lower-frequency, structured or semi-structured records often originating from a separate Computerized Maintenance Management System (CMMS). Because the plants are geographically distributed, connectivity is likely inconsistent (factory-floor networks, occasional outages), which has a strong influence on the probable ingestion design.

2. Probable Integration Tools

- Edge gateways at each plant most likely use an industrial IoT protocol (MQTT or OPC-UA) to collect sensor data locally before forwarding it, since these are the de facto standards for factory-floor telemetry and support lightweight, low-bandwidth publishing suited to unreliable industrial networks.
- A message broker (Kafka, or an MQTT broker bridged into Kafka) aggregates streams from all plants into a centralized ingestion point, likely partitioned by plant ID or machine ID so that downstream consumers can process per-plant or per-machine streams independently.
- An integration/ETL tool (Apache NiFi, or a custom connector) most probably harmonizes the CMMS maintenance-event data (often from a relational database) with the sensor telemetry stream, aligning both onto a common machine-ID and timestamp schema so they can later be joined for predictive-maintenance analysis.
- Given intermittent plant connectivity, a local buffering/store-and-forward mechanism at the edge (so data isn't lost during network outages and is replayed once connectivity resumes) is a strong and defensible inference, since industrial IoT deployments routinely need to tolerate connectivity gaps.

3. Probable Storage Strategy

- Raw, high-frequency sensor telemetry is most likely written to a time-series-optimized store (InfluxDB, or a distributed file system such as HDFS/S3 partitioned by machine and time window), since time-series data has very specific query patterns (range scans over time) that generic relational storage handles poorly at this volume.
- Maintenance events, being lower-volume and structured, are more plausibly kept in a relational or lightly denormalized store, joined against the telemetry data only during analytical processing rather than at ingestion.
- A long-term data lake (HDFS/S3, columnar Parquet format) most likely retains the full historical telemetry and event history across all plants, since predictive-maintenance models need months of prior sensor behavior leading up to past failures to learn meaningful patterns.

4. Probable Analytics Path

- Stream processing (Spark Streaming or Flink) most likely computes rolling statistical baselines per machine (mean/variance of vibration or temperature) and flags real-time deviations, enabling early-warning alerts ahead of full failure.
- A batch ML pipeline, trained on the historical data lake, most likely builds predictive-maintenance models (e.g., survival analysis or classification models predicting failure within N days) by joining sensor trends with historical maintenance/failure records — this join is the core reason integration between the two data types is essential.
- Dashboards aggregating both real-time anomaly flags and model-driven failure-risk scores are most likely served to plant managers through a BI layer, supporting both immediate operational response and longer-term maintenance scheduling.

5. Likely Design Decisions and Rationale

Separating time-series telemetry storage from structured maintenance-event storage is a defensible design decision because the two have very different write patterns (extremely high-frequency small writes vs. low-frequency structured records) that are poorly served by a single storage engine. Introducing edge buffering is a strong, well-justified inference specifically because industrial plants commonly have unreliable network links, and losing sensor data during outages would directly undermine the predictive-maintenance objective.

6. Technical Evidence Supporting This Inference

The explicit mention of data from 'many plants' is evidence of a distributed, per-site ingestion architecture rather than a single centralized collector, since a single collection point would be a scalability and reliability bottleneck across geographically dispersed factories. The combination of machine logs with maintenance events, rather than either alone, is evidence that the ultimate analytics goal is predictive maintenance — correlating sensor drift with historical failures — which is only possible if the two disparate data types are deliberately joined on a common machine/time key.

7. Challenges This Design Most Likely Addresses

- Unreliable factory-floor connectivity: intermittent network drops between plants and the central cloud would silently lose critical sensor readings without edge buffering, which is precisely why store-and-forward at the gateway is a strong and necessary inference rather than an optional enhancement.
- Heterogeneous machine/vendor formats: different machines and plants likely run equipment from multiple vendors with differing telemetry formats, so the integration layer most likely includes a normalization step that maps varied sensor schemas onto a common machine-ID/metric schema before storage.
- Label scarcity for predictive models: actual failure events are relatively rare compared to normal operating data, so the batch ML pipeline most likely leans on techniques suited to imbalanced, rare-event prediction (e.g., survival analysis, anomaly scoring) rather than standard balanced classification.
- Alert fatigue: naively flagging every minor sensor deviation would overwhelm plant technicians with false alarms, so the streaming anomaly-detection layer most likely uses adaptive, per-machine baselines (rather than fixed global thresholds) to reduce false positives and preserve technician trust in the alerts.

8. Trade-offs and Alternative Considerations

A simpler alternative would have every plant write sensor data directly and synchronously into a central relational database. This would simplify the architecture considerably, but it would be fragile against network outages and would not scale to the sustained write volume of continuous multi-plant telemetry, which is why a buffered, streaming-based ingestion path is the more defensible inference. Likewise, relying purely on fixed-threshold alerting (e.g., 'alert if temperature exceeds X') would be simpler to implement than adaptive statistical baselines, but it would generalize poorly across machines with different normal operating ranges — making per-machine adaptive baselines, despite their added modeling complexity, the more plausible and effective design for a platform spanning many different plants and machine types.